Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SOFTWARE ENGINEERING LAB (BCSE301P)

SecuExam (Secure Exam Paper Distribution)

Done by:
AKSHAT SPARSH (23BDS0149)

OTHER TEAM MEMBERS:

TANUJ KAPOOR (23BCT0030)

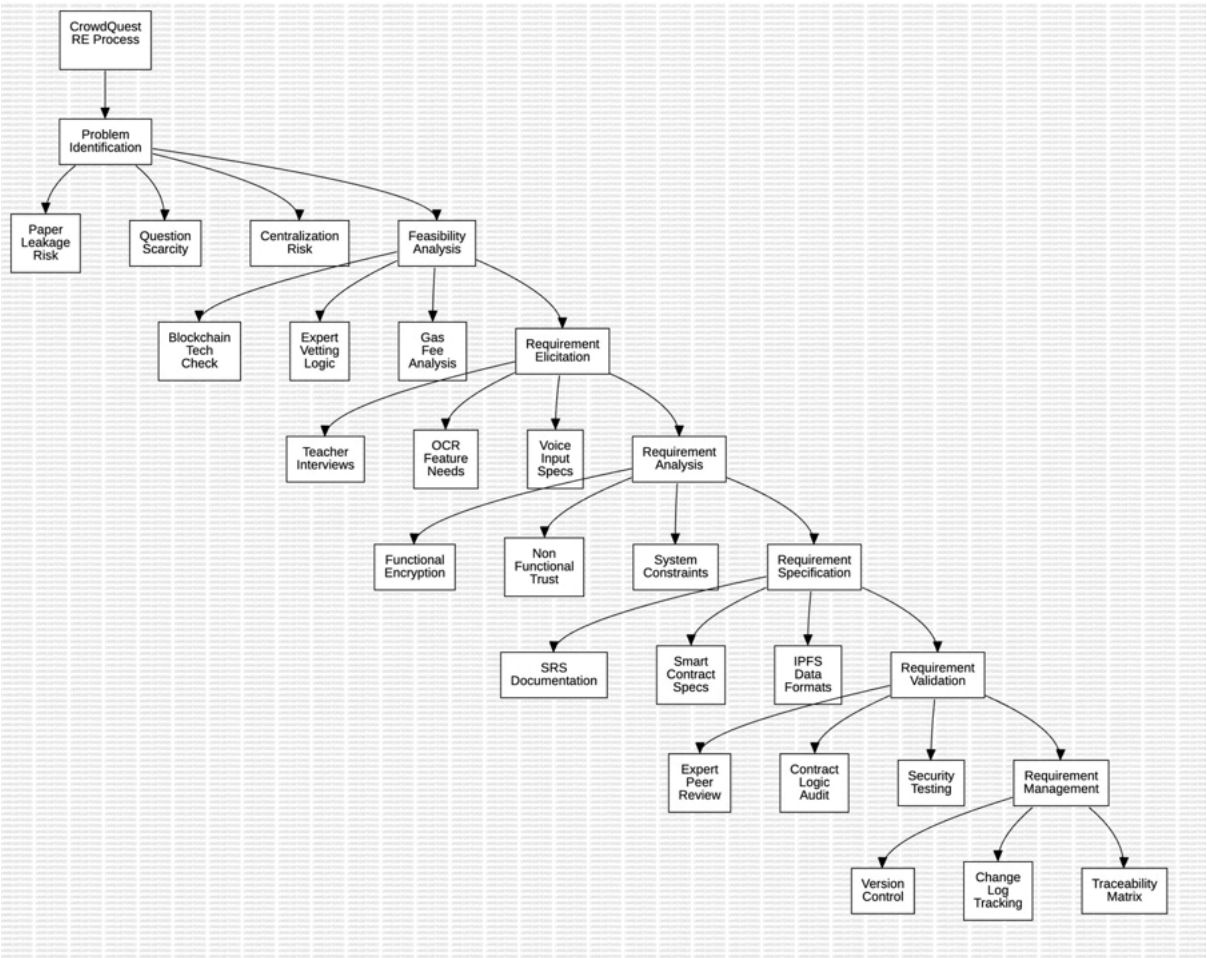
KARTK KANDWAL (23BCT0060)

EXISTING PROJECT: CROWDQUEST

CrowdQuest Project Summary

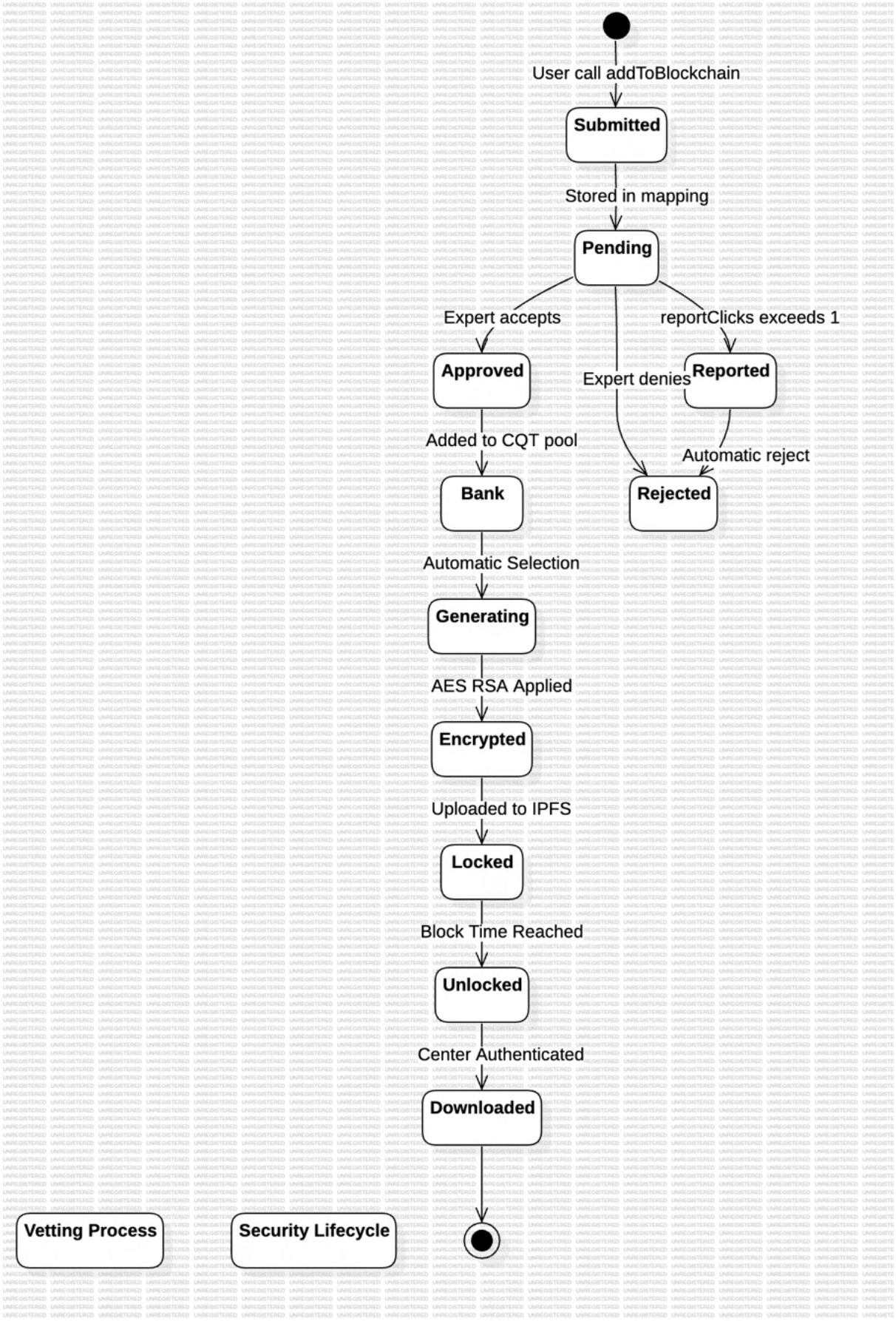
CrowdQuest is a decentralized, blockchain-based examination management platform designed to eliminate exam paper leakage through crowdsourced question generation and secure delivery. The system utilizes **Ethereum/Polygon Blockchain** and **IPFS (InterPlanetary File System)** for storage, ensuring that data is tamper-proof and distributed. The project handles the entire examination lifecycle: contributors submit questions via OCR or voice input, expert teachers vet the pool, the system automatically generates papers, and finally, encrypted PDFs are delivered to centers using a "just-in-time" time-lock mechanism.

RE MODEL DIAGRAM OF CROWDQUEST:



This hierarchical tree maps the systematic process used to define the project's scope. It begins with **Problem Identification** (e.g., paper leakage and centralization risk) and moves through **Feasibility Analysis** (technical blockchain checks and economic gas fee analysis), **Elicitation** (gathering teacher needs), and final **Management** involving traceability matrices and version control.

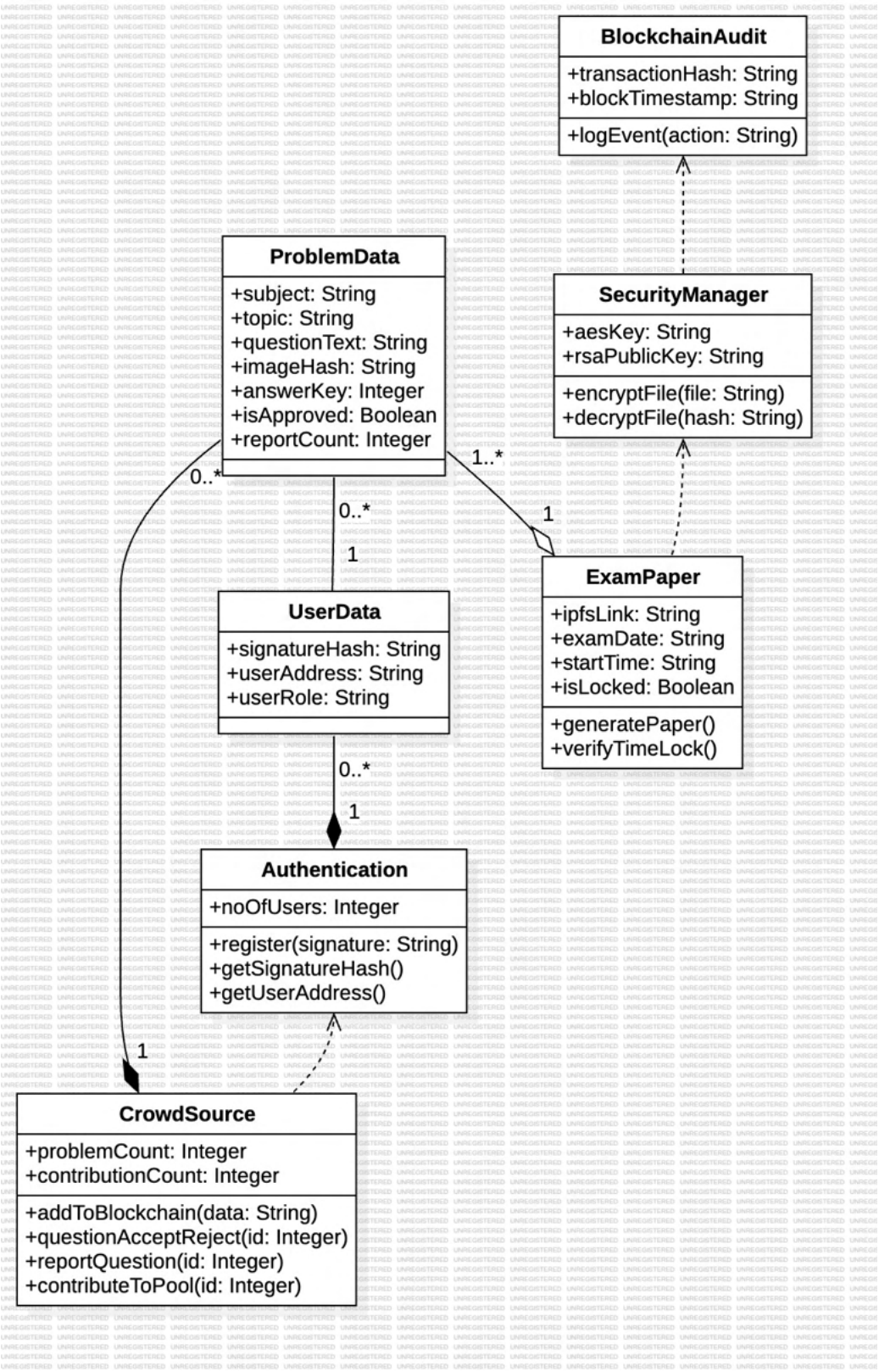
STATE DIAGRAM:



This diagram tracks the lifecycle of system entities through two parallel tracks:

- **Vetting Process:** A question moves from "Submitted" to "Pending" until an expert either "Approves" it into the bank or "Rejects" it (often due to user reports).
- **Security Lifecycle:** An exam paper moves from "Generating" to "Encrypted" and "Locked" on IPFS. It only reaches the "Unlocked" state for center download once the block time is reached.

CLASS DIAGRAM:

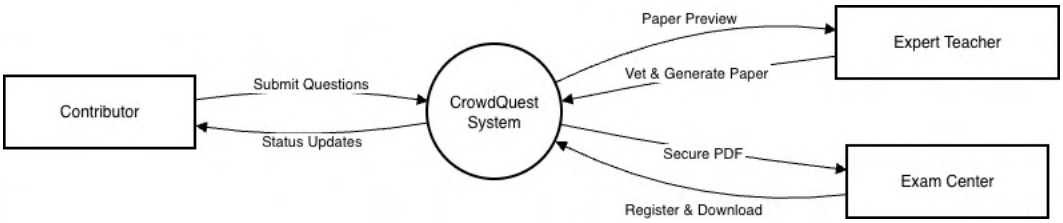


Class Diagram

The class diagram defines the system's static structure and data relationships. Key classes include:

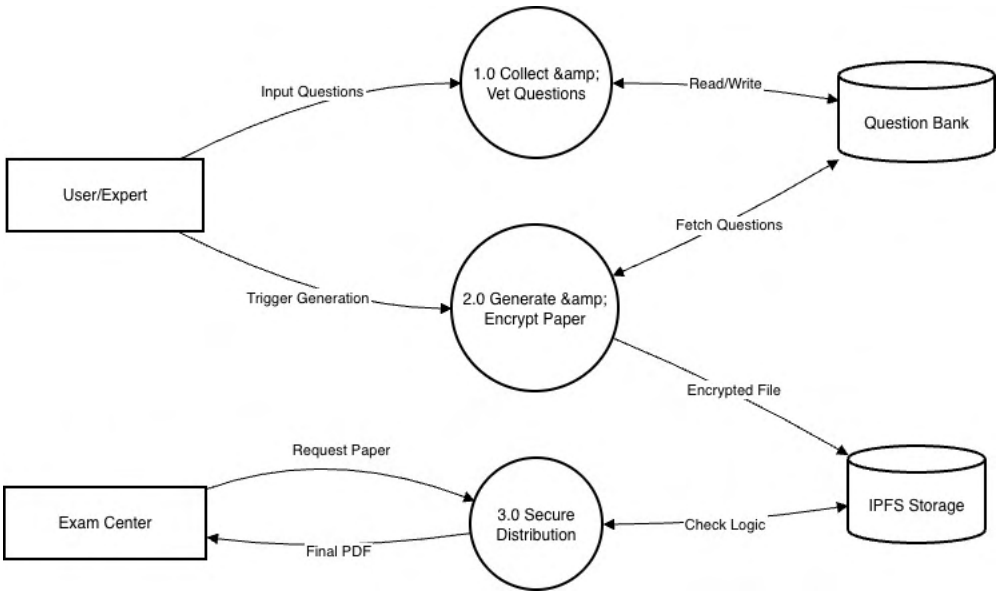
- **Authentication & UserData:** Manage user roles and blockchain signature hashes.
- **CrowdSource & ProblemData:** Handle the logic for question submissions, vetting status, and report counts.
- **SecurityManager & ExamPaper:** Govern AES/RSA encryption and time-lock verification for IPFS-stored papers.
- **BlockchainAudit:** Records transaction hashes and block timestamps for every action.

DFD LEVEL 0:



This high-level flow shows how external actors interact with the central **CrowdQuest System**. **Contributors** submit questions and receive status updates; **Expert Teachers** vet questions and generate papers; and **Exam Centers** register for and download secure PDFs.

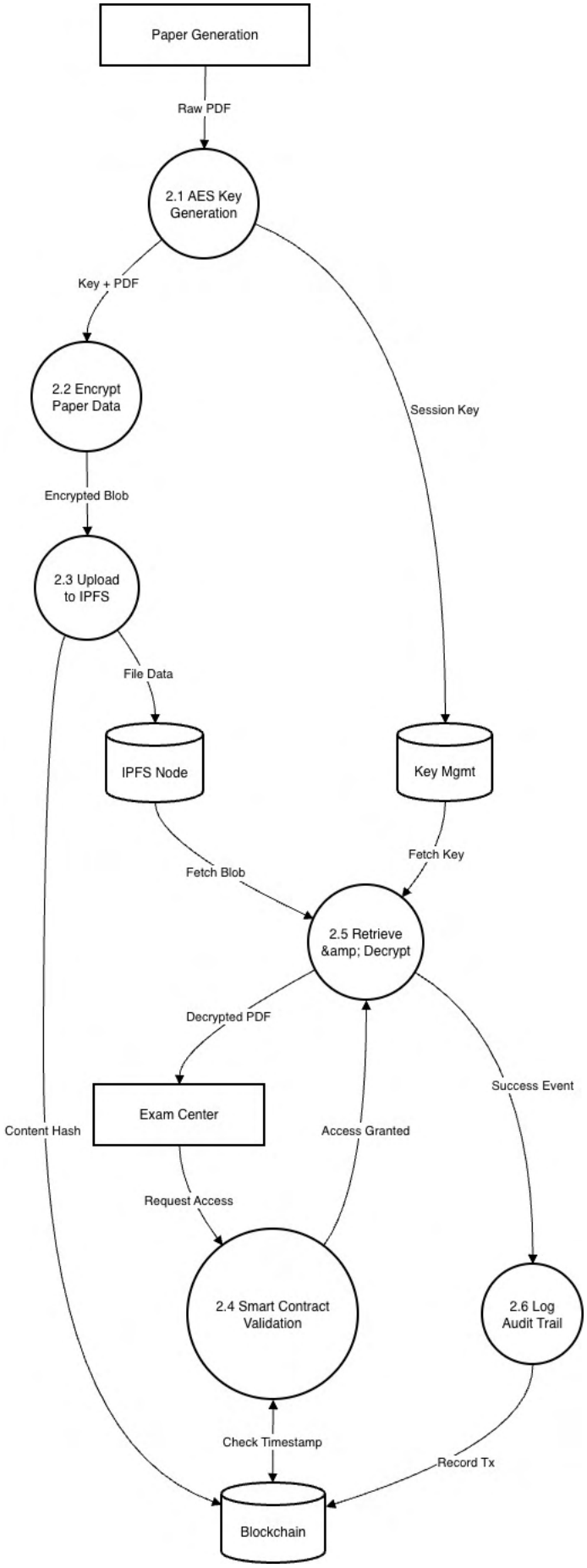
DFD LEVEL 1:



This diagram breaks the system into three core functional processes:

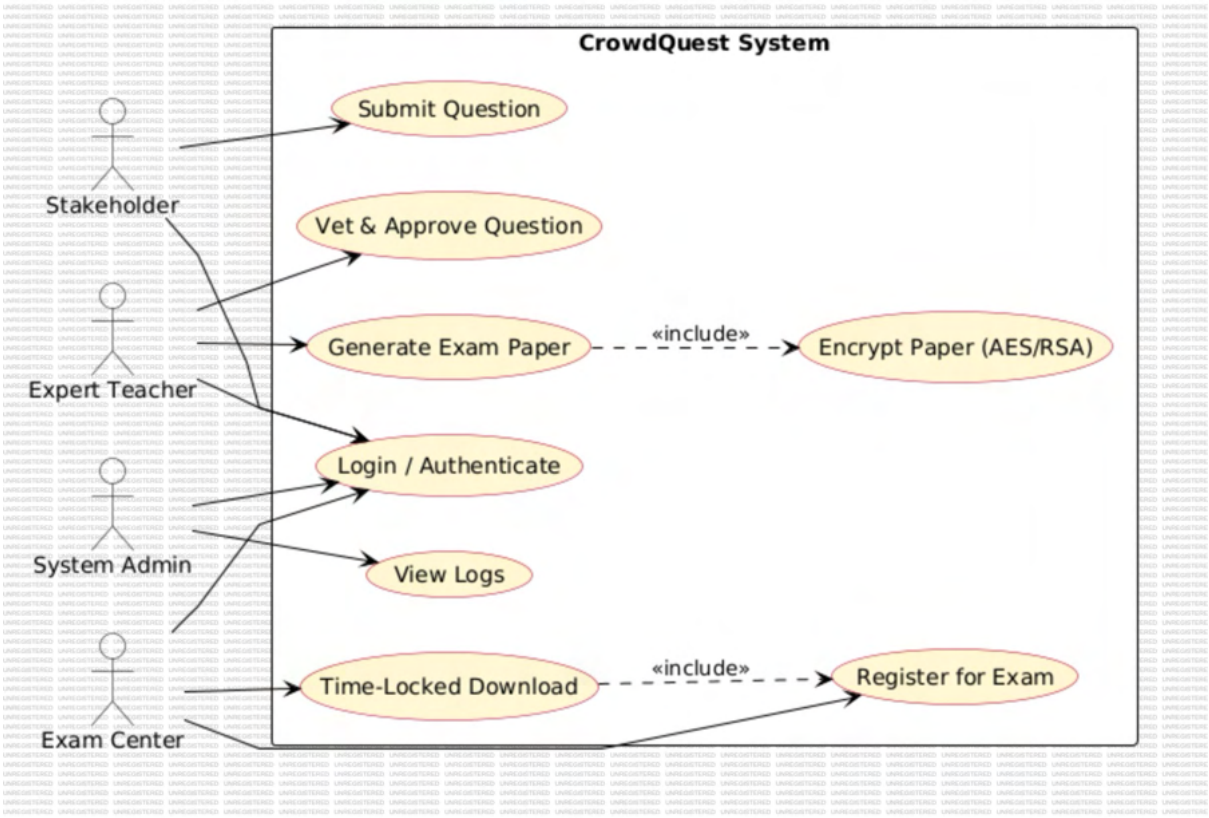
1. **Collect & Vet Questions:** Interacts with the **Question Bank** data store.
2. **Generate & Encrypt Paper:** Fetches vetted questions and stores the encrypted result in **IPFS Storage**.
3. **Secure Distribution:** Manages center requests and validates download logic.

DFD LEVEL 2:


Paper Generation
Raw PDF
2.1 AES Key Generation
Key + PDF
Session Key
2.2 Encrypt Paper Data
Encrypted Blob
2.3 Upload to IPFS
File Data
IPFS Node
Key Mgmt
Fetch Key
Fetch Blob
2.5 Retrieve & Decrypt
Decrypted PDF
Success Event
Exam Center
Content Hash
Access Granted
Request Access
2.4 Smart Contract Validation
2.6 Log Audit Trail
Check Timestamp
Record Tx
Blockchain

A granular view of the distribution module, showing the cryptographic pipeline. It details **AES Key Generation**, the splitting of session keys, the uploading of encrypted "blobs" to **IPFS**, and the **Smart Contract Validation** that checks timestamps against the blockchain before allowing a center to decrypt the final PDF.

UML USE CASE DIAGRAM:

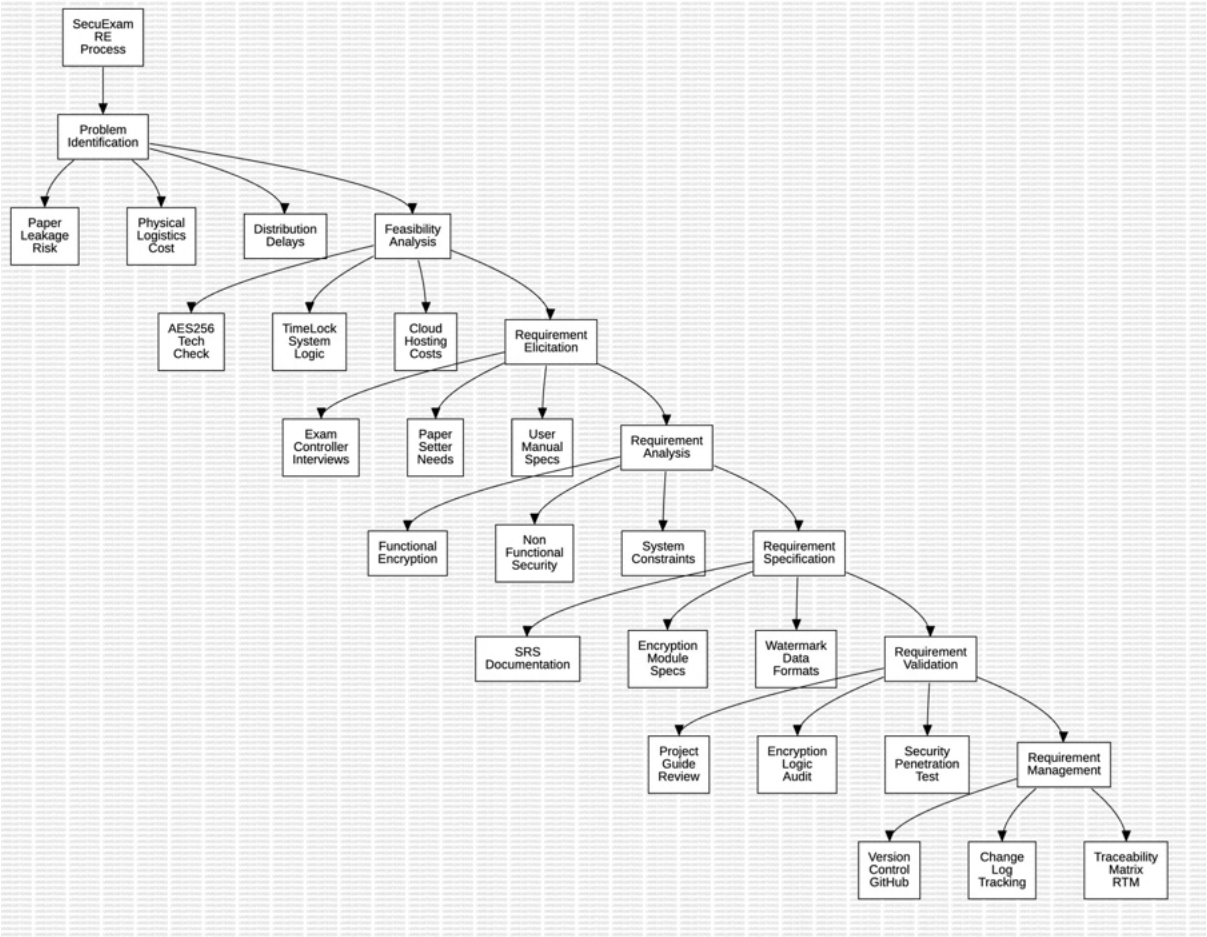


This diagram identifies the specific actions actors can take. For example, the **Expert Teacher** "vets questions" and "generates exam papers," while the **Exam Center** performs "time-locked downloads". It highlights critical dependencies, such as paper generation automatically "including" AES/RSA encryption.

OUR PROJECT: SECUEXAM

SecuExam is a secure exam paper distribution system designed to eliminate paper leakage. The system replaces traditional physical logistics with a digital, high-security workflow. It relies on **AES-256 encryption** for data protection and **Shamir's Secret Sharing** for key management. Key security features include **Time-Locking**, which prevents access before the scheduled exam time, and **Dynamic Watermarking**, which embeds the receiver's IP onto the document to track any unauthorized sharing.

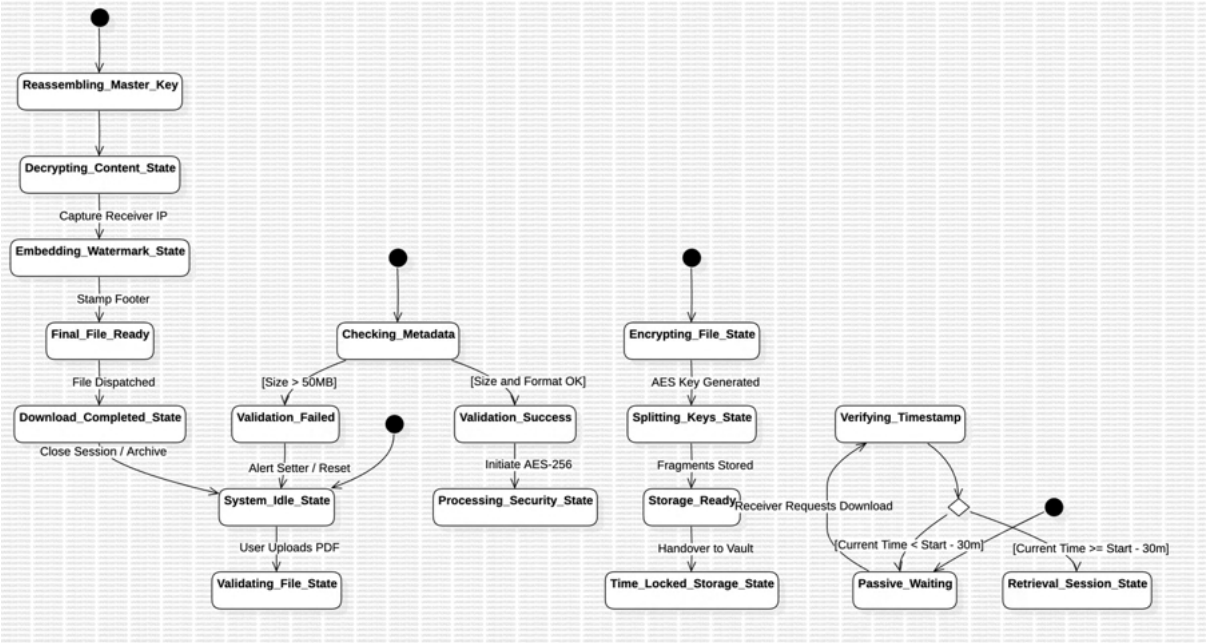
RE MODEL DIAGRAM:



This hierarchical flowchart maps out the entire project lifecycle across seven phases:

- **Problem Identification & Feasibility:** Highlights risks like paper leakage and physical logistics costs while analyzing the technical feasibility of AES-256 and Time-Locking .
- **Elicitation & Analysis:** Details gathering requirements from Exam Controllers and Setters to define functional encryption and non-functional security needs .
- **Specification, Validation, & Management:** Outlines the creation of the SRS document, project guide reviews, security penetration testing, and the use of a **Requirements Traceability Matrix (RTM)** for version control on GitHub .

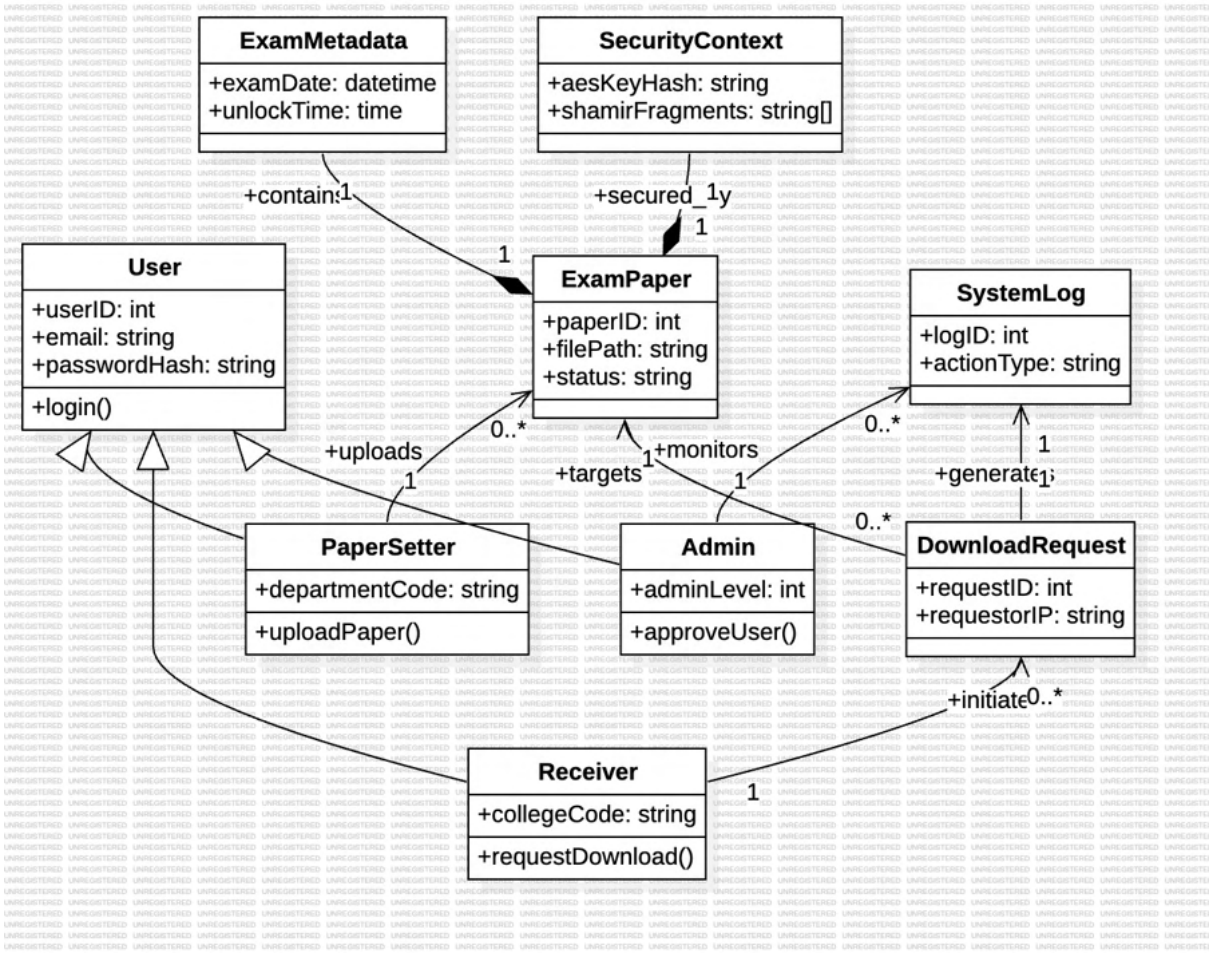
STATE DIAGRAM:



This diagram illustrates the lifecycle of an exam paper as it moves through various system states:

- **Upload & Security:** An uploaded PDF is validated for size and format before moving to an **Encrypting File State** (AES-256) and a **Splitting Keys State** (Shamir's) .
- **Storage & Access:** The paper enters a **Time Locked Storage State** where it passively waits. A download request triggers a **Verifying Timestamp** state; if it is too early, the system denies access.
- **Retrieval:** Once authorized, the system enters the **Retrieval Session State** to reassemble keys, decrypt content, and embed watermarks before final delivery .

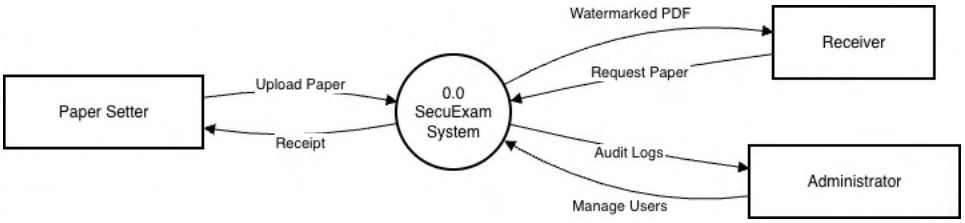
CLASS DIAGRAM:



The class diagram defines the system's data structure and object relationships:

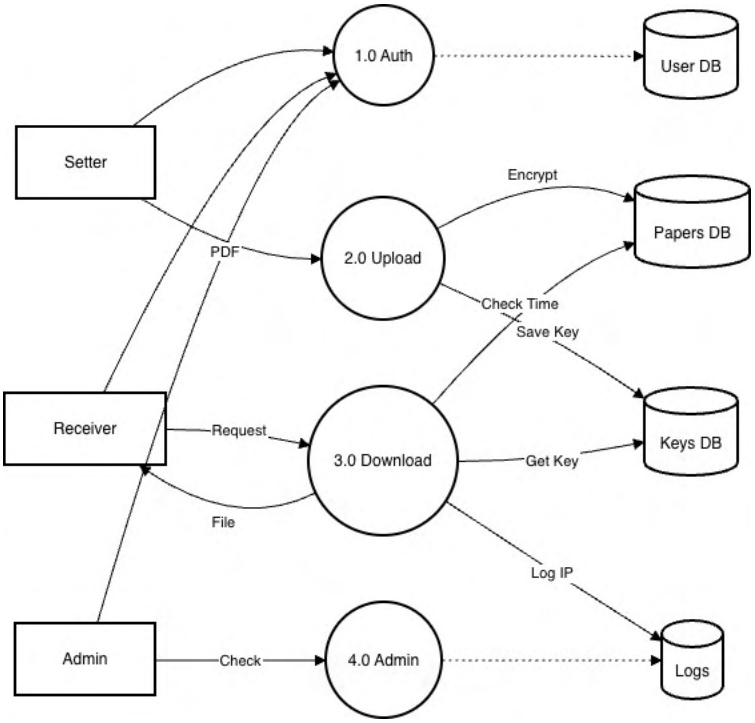
- **User Hierarchy:** A base User class is inherited by specialized roles: PaperSetter, Receiver, and Admin.
- **ExamPaper Composition:** An ExamPaper object is composed of ExamMetadata (date/time) and a SecurityContext (AES hashes and Shamir fragments) .
- **Tracking:** DownloadRequest objects are generated by receivers and monitored by the Admin via SystemLog entries to ensure full auditability.

DFD LEVEL 0:



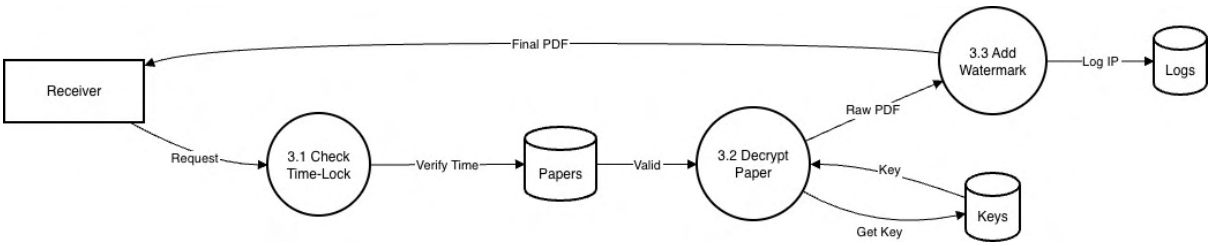
Shows high-level interactions between the **SecuExam System** and its three primary actors: Paper Setter, Receiver, and Administrator.

DFD LEVEL 1:



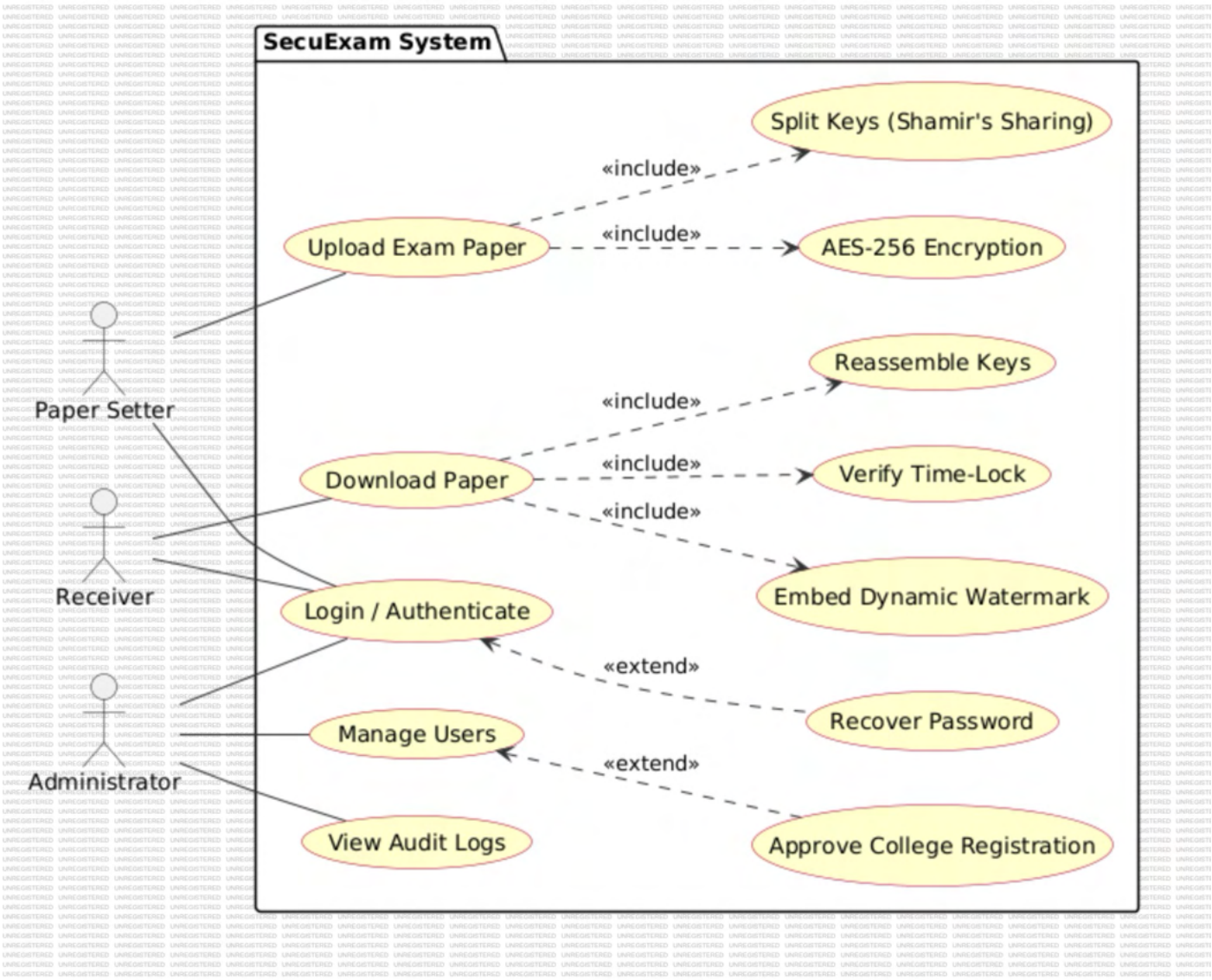
Breaks the system into four processes: **1.0 Auth**, **2.0 Upload**, **3.0 Download**, and **4.0 Admin**, interacting with databases for Users, Papers, Keys, and Logs .

DFD LEVEL 2:



Specifically drills into the download logic, detailing the sequence of **Checking the Time-Lock**, **Decrypting the Paper** with retrieved keys, and **Adding the Watermark** .

USE CASE:



This diagram identifies the actions each user can perform within the system boundary:

- **Paper Setter:** Can upload papers, which automatically triggers (includes) **Split Keys** and **AES-256 Encryption** .
- **Receiver:** Can download papers, which requires the system to **Reassemble Keys**, **Verify Time-Lock**, and **Embed Dynamic Watermark** .
- **Administrator:** Manages users and views audit logs, with the ability to approve college registrations .